

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	18
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	19
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	20
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	21
8.2.3.5	is_busy()	22
8.2.3.6	stop()	22
8.2.3.7	add_action()	22
8.2.3.8	attach()	23
8.2.3.9	detach()	24
8.2.3.10	results()	25
8.3	untangle::async::execution_poll Class Reference	25
8.3.1	Detailed Description	26
8.3.2	Member Function Documentation	26
8.3.2.1	add()	26
8.3.2.2	remove()	26

8.3.2.3 <code>is_running()</code>	27
8.3.2.4 <code>get()</code>	27
8.4 <code>untangle::async::invalid_attachment</code> Struct Reference	27
8.4.1 Detailed Description	27
8.4.2 Constructor & Destructor Documentation	27
8.4.2.1 <code>invalid_attachment()</code>	27
8.4.3 Member Function Documentation	28
8.4.3.1 <code>what()</code>	28
9 File Documentation	29
9.1 <code>async.hpp</code>	29
Index	37

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
action();
// run the execution
execution->run();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(*execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

```

```

}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The actuator submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Execution poll class	25
untangle::async::invalid_attachment	
Invalid attachment exception	27

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		29
---------------------------	-------	--------------------

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

Deliberately at namespace scope rather than nested in [execution](#), so that it is one type rather than one per specialisation. An execution may be attached by an execution of any specialisation, and the whole point of this record is that the chain can be walked without knowing what any link's action type is.

An execution holds one of these by `std::shared_ptr` and it dies with the execution, so it doubles as the liveness token the attached side needs: a link whose weak reference still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- [execution](#) ()
Constructs a new execution object with the default name. Call [create_instance\(\)](#) instead.
- [execution](#) (std::string exec_name)
Constructs a new named execution object.
- [~execution](#) ()
Destroys the execution object, once its worker has left.
- bool [is_running](#) () const
Checks if this execution has finished.
- bool [is_busy](#) () const
Is this execution working through its action list?
- void [stop](#) ()
Stops the worker, after the actions already queued have run.
- template<typename... Args>
bool [add_action](#) (actionT action, Args... args)
Queues an action for this execution's worker, and says whether it was taken.
- template<typename otherT>
void [attach](#) (const std::shared_ptr< otherT > &other)
Attaches another execution, so that this one triggers it.
- template<typename otherT>
bool [detach](#) (otherT &other)
Detaches an execution previously attached by [attach\(\)](#).
- auto [results](#) () const
The return values of the actions run by [run\(\)](#), in the order they ran.

Static Public Member Functions

- template<typename... Args>
static std::shared_ptr< execution > [create_instance](#) (Args &&... args)
Creates an execution owned by a std::shared_ptr, which is what binding to it requires.
- template<typename classT, typename T>
static void [bind_action_and_method](#) (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a class function member.
- template<typename T>
static void [bind_action_and_function](#) (actionT &action, const T &Fn, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a plain function.

Static Public Attributes

- static constexpr auto default_name = "default_name"
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Template Parameters

actionT	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
---------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 execution() [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs a new execution object with the default name. Call [create_instance\(\)](#) instead.

The name identifies an execution in the warnings it reports, and is otherwise unused. An execution built without one keeps the default name rather than an empty one, so a warning always names something.

8.2.2.2 execution() [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a new named execution object.

An execution constructed directly cannot be bound to [bind_action_and_method\(\)](#) and [bind_action_and_function\(\)](#) require a `std::shared_ptr`. Use [create_instance\(\)](#) for one that is going to carry bound actions.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution object, once its worker has left.

The worker is detached and cannot be joined, so this waits on the same "running" state that [execution_poll](#) reports - the only handle a detached worker offers. It returns at once for an execution that was never started, or that the caller has already polled to a stop.

Remarks

It first takes this execution out of whatever attached it, which is the inverse of [attach\(\)](#) and the counterpart of the [execution_poll::remove\(\)](#) below.

This is safe only because `running_` is cleared as the very last thing the worker does. Nothing may be added after it in `execute()` or `loop()`: the object can be freed the moment it reads false.

No rule of five, deliberately. An execution is declared once and used in place; it is never copied, moved or assigned. Declaring this destructor suppresses the implicit moves, and the copy operations are deleted several times over - by `std::thread`, by every atomic, by both mutexes and by the condition variable, each on its own. Spelling the four out would only restate that, and would start earning its keep only if all of those members were replaced by copyable ones at once. `execution_` [↩](#) `special_members.cannot_be_copied_or_moved` pins the result.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.

`bind_action_and_method()` and `bind_action_and_function()` take the execution as a `std::shared_ptr` and hold it weakly, so an execution that is to be bound has to be created this way. One built on the stack, as a data member, or through a bare new is still a usable execution

- actions can be handed to `add_action()` directly - it simply cannot be bound, and the attempt does not compile.

The arguments are forwarded to a constructor, so this factory does not have to be revisited when one is added.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A `std::shared_ptr` owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by `untangle::bind()`), and assigns to action a callable that passes it to `add_action()`. The execution is taken as a `std::shared_ptr` and held as a `std::weak_ptr`, exactly as `untangle::bind()` holds the bound object: the action can therefore outlive the execution and report a dead binding rather than following a dangling reference.

Attention

Invoking action after the execution has been destroyed throws `untangle::invalid_action`. `execute↵actions()` catches it, so such an action is dropped with a warning rather than ending the worker thread.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
obj	- A <code>std::shared_ptr</code> that wraps the bound class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code> .
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on. Requiring it here is what keeps an execution that no <code>shared_ptr</code> owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    const T & Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps `Fn` in an action and assigns to `action` a callable that passes it to `add_action()`. The execution is held weakly, for the reason given on `bind_action_and_method()`.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
Fn	- A plain function.
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on.

8.2.3.4 is_running()

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Checks if this execution has finished.

Returns

- true - The execution has not finished.
- false - The execution has finished.

Is this execution still working?

False from the moment the worker starts reporting itself finished, which is before `running_` is cleared - the two answer different questions. This one is for callers, and says whether there is still work going on; `running_` is the handshake `~execution()` waits on, and says whether the worker is still touching this object. A caller polling this one therefore learns that a run is over without having to wait for the object to be safe to destroy.

Remarks

A callback is not such a caller. `on_finished` is raised from inside the drain, on the worker's own thread, and this reads true there - the worker is standing in the callback with the rest of its path still to go. That a batch drained says nothing about the worker on either path, and is not meant to.

8.2.3.5 `is_busy()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]
```

Is this execution working through its action list?

True while anything is queued, and while an action taken off the queue is still running - the gap between those two is why this is not simply a test for an empty list. An execution is busy from the moment an action is added until the last one has returned.

Distinct from [is_running\(\)](#), which is about the worker thread. A continuous worker started by `start()` is running for its whole life whether or not it has anything to do, so [is_running\(\)](#) cannot answer "has this one got room for more work"; this can. A one-shot `run()` is both.

Remarks

Answers for this execution's own actions. An execution that is triggering others through [attach\(\)](#) reports on its own list, not on theirs - see item 12.

A true answer is a fact about the instant it was taken, and the execution may drain the moment after. A false answer is durable only if the caller is the one adding the actions, which is the case this is written for: asking whether a worker has room before handing it more.

Returns

true - actions are queued, or one is running.

8.2.3.6 `stop()`

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to [add_action\(\)](#) after this call is refused, not queued: [stop\(\)](#) is the end of this execution's working life, and an action accepted then would never run.

8.2.3.7 `add_action()`

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```

Queues an action for this execution's worker, and says whether it was taken.

The action is bound to args here and runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before `run()` or `start()` is normal; the actions already in the list are what the worker drains first.

Attention

An execution that has been stopped refuses. `stop()` ends its working life, so an action accepted afterwards would sit in the list looking pending and never run. The refusal is the return value, and a caller that ignores it loses the action silently - a warning on stderr is what this used to offer instead, and it is no substitute for an answer.

Remarks

A refusal is not `untangle::invalid_action`, and is deliberately not reported as one. That exception means a binding whose target has died - the action itself is broken. Here the action is sound and the execution is simply closed to new work, which is an answer, not a fault.

A caller arriving through `bind_action_and_method()` or `bind_action_and_function()` does not see this. Those lambdas return `actionT::result_type`, fixed by the specialisation, so there is nowhere to put a bool; they still swallow the outcome. Telling that caller is unfinished business, and the same question as what becomes of an action whose body throws.

Template Parameters

Args	- The argument types the action is bound to.
------	--

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

true - The action was queued and this execution's worker will run it.
false - The execution is stopped and the action was dropped.

8.2.3.8 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too.

The actuators are given other's own actions, so the attachment is a pointer into other. What makes that safe is the record left on the other side: other is told which actuators hold it, and `~execution()` takes itself back out of them. The attacher's lifetime token guards those pointers, so an attached execution outliving its attacher follows nothing.

Parameters

other	- A <code>std::shared_ptr</code> owning the execution to trigger. Requiring it here is what keeps an execution that no <code>std::shared_ptr</code> owns from being attached at all, exactly as bind_action_and_method() requires one to bind. Attaching the same execution twice adds it twice; a single detach() still removes it completely, because <code>untangle::actuator::remove()</code> erases every match.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which is what keeps the attachment graph a forest and the cycle check below a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, which includes attaching an execution to itself. Driving a cycle recurses until the stack is gone, so it is refused here, where the caller still has a stack to be told on.

8.2.3.9 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by [attach\(\)](#).

Both paths are unwired, the triggering one and the stopping one: an attachment that could only be half undone would leave this execution still able to stop one it no longer drives.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

true - other was attached and is no longer.
false - other was not attached to this execution.

8.2.3.10 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]
```

The return values of the actions run by run(), in the order they ran.

Read it from on_finished, which fires once the queue has drained and before the execution reports itself finished, so the results are complete by the time the callback can see them.

Returned by value, under results_mutex_ - the worker appends to the vector as it goes, so handing out a reference would hand out something being written.

Remarks

Only run() fills this. The continuous worker started by start() does not: it has no point at which a run is over, so there is nothing to hand back and nowhere to clear, and filling it would grow without bound. Deferred until there is a use case that wants it.

Returns

The results of the most recent run, or empty if none has produced any.

The documentation for this class was generated from the following file:

- async.hpp

8.3 untangle::async::execution_poll Class Reference

Execution poll class.

```
#include <async.hpp>
```

Public Member Functions

- template<typename asyncexecT>
void add (asyncexecT &async_exec)
Adds an execution object to the poll.
- template<typename asyncexecT>
void remove (asyncexecT &async_exec)
Removes an execution object from the poll.
- auto is_running ()
Checks if the execution objects are running.

Static Public Member Functions

- static execution_poll & get ()
Gets the single-tone instance.

8.3.1 Detailed Description

Execution poll class.

A single-tone class that may be used to verify if the polled [execution](#) objects have finished their processing. If at least one [execution](#) object is running then this poll has the "running" state.

Remarks

An [execution](#) object runs in a detached thread, hence it is not joinable. The caller can wait for an [execution](#) to be finished only by checking the "running" state([execution::is_running\(\)](#)). This class represents a convenient way to check any number of [execution](#) objects.

8.3.2 Member Function Documentation

8.3.2.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

[add\(\)](#) stores a pointer to the object's `action_is_running`, so an execution destroyed while still registered leaves the poll calling into freed memory. `~execution()` calls this.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.3 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

8.3.2.4 get()

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.4 untangle::async::invalid_attachment Struct Reference

Invalid attachment exception.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Invalid attachment exception.

Remarks

Raised by [execution::attach\(\)](#) when the attachment asked for would close a cycle - either between two executions that each attach the other, or an execution attached to itself. Driving such a cycle recurses until the stack is gone, so it is refused where the caller can still be told about it.

Deliberately not an [untangle::invalid](#) action. That one reports a binding whose target has died, and by [execution::execute_actions\(\)](#) swallows it by design; a cycle is a caller error and must not be swallowed.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 what()

```
const char * untangle::async::invalid_attachment::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- async.hpp

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <condition_variable>
00012 #include <exception>
00013 #include <memory>
00014 #include <mutex>
00015 #include <print>
00016 #include <string>
00017 #include <thread>
00018 #include <vector>
00019
00020 namespace untangle {
00021 template <typename actionT>
00022 struct actuator;
00023
00024 namespace async {
00025 template <typename actionT>
00026 class execution;
00027 }
00028 } // namespace untangle
00029
00030 namespace untangle {
00031 namespace async {
00032
00035 struct invalid_attachment : std::exception {
00054     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00055
00061     const char* what() const noexcept override { return message_.c_str(); }
00062
00063 private:
00064     std::string message_;
00065 };
00066
00079 struct attachment {
00081     std::weak_ptr<attachment> attacher;
00082 };
00083
00098 class execution_poll {
00099 public:
00105     template <typename asyncexecT>
00106     void add(asyncexecT& async_exec) {
00107         std::lock_guard<std::mutex> lock(actuator_mutex_);
00108
00109         if (!actuator_is_running_.is_connected()) {
00110             actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00111         } else {
00112             actuator_is_running_.add(&async_exec.action_is_running);
00113         }
00114     }
00115
00124     template <typename asyncexecT>
00125     void remove(asyncexecT& async_exec) {
```

```

00126     std::lock_guard<std::mutex> lock(actuator_mutex_);
00127
00128     actuator_is_running_.remove(&async_exec.action_is_running);
00129 }
00130
00136 auto is_running() {
00137     std::lock_guard<std::mutex> lock(actuator_mutex_);
00138     actuator_is_running_();
00139
00140     auto result = false;
00141     for (const auto& ret : actuator_is_running_.results) {
00142         result |= ret;
00143     }
00144     return result;
00145 }
00146
00152 static execution_poll& get() {
00153     static execution_poll instance;
00154     return instance;
00155 }
00156
00157 private:
00158     execution_poll() = default;
00159     ~execution_poll() = default;
00160     actuator<std::function<bool(void)>> actuator_is_running_;
00161     mutable std::mutex actuator_mutex_;
00162 };
00163
00177 template <typename actionT>
00178 class execution {
00179     // An execution may attach one of a different specialisation - the smoke test attaches an
00180     // execution<function<void(int)>> to an execution<function<void(void)>> - and that is a different
00181     // class with no access to this one's members. attach() and detach() need to read and write
00182     // attachment_lifetime_ on it.
00183     template <typename otherActionT>
00184     friend class execution;
00185
00186 public:
00203     template <typename... Args>
00204     static std::shared_ptr<execution> create_instance(Args&&... args) {
00205         return std::make_shared<execution>(std::forward<Args>(args)...);
00206     }
00207
00216     execution() : execution(std::string(default_name)) {}
00217
00227     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00228         other_this_ = this;
00229         action_execute = untangle::bind(other_this_, &execution<actionT>::execute_actions);
00230         action_stop = untangle::bind(other_this_, &execution<actionT>::stop);
00231         action_is_running = untangle::bind(other_this_, &execution<actionT>::is_running);
00232     }
00233
00255     ~execution() {
00256         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00257         // may reach this object, and the actions about to be destroyed are the ones it holds.
00258         if (!attachment_lifetime_->attacher.expired()) {
00259             attacher_execute_->remove(&action_execute);
00260             attacher_stop_->remove(&action_stop);
00261         }
00262
00263     {
00264         std::lock_guard<std::mutex> lock(action_mutex_);
00265         started_ = false;
00266         stopped_ = true;
00267     }
00268
00269     action_cv_.notify_all();
00270
00271     while (running_.load()) { // time of check
00272         std::this_thread::sleep_for(std::chrono::milliseconds(1));
00273     }
00274
00275     // time of use: this object is freed once the destructor returns, so the check above is only
00276     // safe because running_ is the last thing the worker touches
00277     execution_poll::get().remove(*this);
00278 }
00279
00300 template <typename classT, typename T>
00301 static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00302                                     T classT::* method,
00303                                     const std::shared_ptr<execution>& async_exec) {
00304     actionT async_action = untangle::bind(obj, method);
00305
00306     action = [wp = std::weak_ptr<execution>(async_exec),
00307              async_action](auto... args) -> actionT::result_type {
00308         // lock() also keeps the execution alive for the duration of the call
00309         const auto exec = wp.lock();

```

```

00310     if (!exec) {
00311         throw invalid_action("bind: invalid execution");
00312     }
00313     exec->add_action(async_action, args...);
00314     return typename actionT::result_type();
00315 };
00316 }
00317
00329 template <typename T>
00330 static void bind_action_and_function(actionT& action, const T& Fn,
00331                                     const std::shared_ptr<execution>& async_exec) {
00332     actionT async_action = Fn;
00333
00334     action = [wp = std::weak_ptr<execution>(async_exec),
00335             async_action](auto... args) -> actionT::result_type {
00336         const auto exec = wp.lock();
00337         if (!exec) {
00338             throw invalid_action("bind: invalid execution");
00339         }
00340         exec->add_action(async_action, args...);
00341         return typename actionT::result_type();
00342     };
00343 }
00344
00365 bool is_running() const { return running_.load() && !finishing_.load(); }
00366
00387 bool is_busy() const {
00388     std::lock_guard<std::mutex> lock(action_mutex_);
00389     return !action_list_.empty() || executing_action_.load();
00390 }
00391
00392 void run() {
00393     running_ = true;
00394     finishing_ = false;
00395     collecting_results_ = true;
00396     this_thread_ = std::thread(&execution::execute, this);
00397     this_thread_.detach();
00398 }
00399
00400 void start() {
00401     running_ = true;
00402     finishing_ = false;
00403     collecting_results_ = false;
00404     {
00405         std::lock_guard<std::mutex> lock(action_mutex_);
00406         started_ = true;
00407         stopped_ = false;
00408     }
00409
00410     this_thread_ = std::thread(&execution::loop, this);
00411     this_thread_.detach();
00412 }
00413
00420 void stop() {
00421     {
00422         std::lock_guard<std::mutex> lock(action_mutex_);
00423         started_ = false;
00424         stopped_ = true;
00425     }
00426
00427     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00428     // spin on the list; waking it is enough.
00429     action_cv_.notify_all();
00430
00431     if (actuator_stop_.is_connected()) {
00432         actuator_stop_();
00433     }
00434 }
00435
00466 template <typename... Args>
00467 bool add_action(actionT action, Args... args) {
00468     {
00469         std::lock_guard<std::mutex> lock(action_mutex_);
00470
00471         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00472         // here is what keeps it from sitting in the list looking as though it were pending.
00473         if (stopped_) {
00474             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00475             return false;
00476         }
00477
00478         action_list_.push_back(std::bind(action, args...));
00479     }
00480
00481     action_cv_.notify_one();
00482     return true;
00483 }

```

```

00484
00510 template <typename otherT>
00511 void attach(const std::shared_ptr<otherT>& other) {
00512     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00513         throw invalid_attachment("attach: an execution cannot be attached to itself");
00514     }
00515
00516     if (!other->attachment_lifetime_->attacher.expired()) {
00517         throw invalid_attachment("attach: that execution is attached already");
00518     }
00519
00520     // Refusing an execution that has an attacher already leaves every execution with at most one,
00521     // so the graph is a forest and this new edge closes a cycle exactly when other is somewhere up
00522     // this execution's own chain of attachers. Walking it is the whole check.
00523     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00524         if (link == other->attachment_lifetime_) {
00525             throw invalid_attachment("attach: the attachment would close a cycle");
00526         }
00527     }
00528
00529     if (!actuator_execute_.is_connected()) {
00530         actuator_execute_ = untangle::connect(other->action_execute);
00531     } else {
00532         actuator_execute_.add(&other->action_execute);
00533     }
00534
00535     if (!actuator_stop_.is_connected()) {
00536         actuator_stop_ = untangle::connect(other->action_stop);
00537     } else {
00538         actuator_stop_.add(&other->action_stop);
00539     }
00540
00541     // What ~execution() needs to take itself back out of these actuators: the link proves this
00542     // attacher is still there, and the two pointers are only ever followed while it does.
00543     other->attachment_lifetime_->attacher = attachment_lifetime_;
00544     other->attacher_execute_ = &actuator_execute_;
00545     other->attacher_stop_ = &actuator_stop_;
00546 }
00547
00560 template <typename otherT>
00561 bool detach(otherT& other) {
00562     // The actuator's own action list is the record of what is attached; nothing else has to keep
00563     // one. An action is stored as a pointer, so the attachment is found by identity.
00564     const auto& actions = actuator_execute_.actions;
00565     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00566         return false;
00567     }
00568
00569     actuator_execute_.remove(&other.action_execute);
00570     actuator_stop_.remove(&other.action_stop);
00571
00572     other.attachment_lifetime_->attacher.reset();
00573     other.attacher_execute_ = nullptr;
00574     other.attacher_stop_ = nullptr;
00575
00576     return true;
00577 }
00578
00594 auto results() const {
00595     std::lock_guard<std::mutex> lock(results_mutex_);
00596     return results_;
00597 }
00598
00599 static constexpr auto default_name = "default_name";
00600
00601 std::function<void(void)> action_execute;
00602 std::function<void(void)> action_stop;
00603 std::function<bool(void)> action_is_running;
00604 std::function<void(void)> on_finished;
00605 std::string name = default_name;
00606
00607 private:
00608     using queued_action_t = std::function<typename actionT::result_type(void)>;
00609
00618 void execute_action(queued_action_t& action) {
00619     if constexpr (std::is_void_v<typename actionT::result_type>) {
00620         action();
00621     } else {
00622         auto value = action();
00623
00624         if (collecting_results_) {
00625             std::lock_guard<std::mutex> lock(results_mutex_);
00626             results_.push_back(std::move(value));
00627         }
00628     }
00629 }
00630

```

```

00645 void execute_actions() {
00646     actions_run_ = 0;
00647
00648     // The action is taken off the list under the lock and invoked with the lock released: an action
00649     // is caller code that may run for a while, and may itself call add_action().
00650     for (;;) {
00651         queued_action_t action;
00652
00653         {
00654             std::lock_guard<std::mutex> lock(action_mutex_);
00655             if (action_list_.empty()) {
00656                 break;
00657             }
00658
00659             action = std::move(action_list_.front());
00660             action_list_.pop_front();
00661
00662             // Set here, under the lock that emptied the list, so that there is no instant in which the
00663             // list reads empty while this action has not yet run. is_busy() takes the same lock, so it
00664             // sees the pop and this together or neither.
00665             executing_action_ = true;
00666         }
00667
00668         // Nothing may leave this loop. The worker is a detached thread function, so an exception
00669         // escaping it calls std::terminate and gives no one anywhere to catch it - and an execution
00670         // exists to run code the header did not write, which can throw anything at all.
00671         //
00672         // Two kinds arrive here. invalid_action is the header's own: a binding whose target has died,
00673         // which the actuator drops and so does this. Anything else came out of the action's body and
00674         // is the caller's, and the answer is the same - name the execution, drop the action, and go
00675         // on to the next one. A worker that died with the first task that threw could not be given
00676         // work it did not write.
00677         try {
00678             execute_action(action);
00679         } catch (const invalid_action& ia) {
00680             std::println(stderr, "warning: execution '{}', dropped an invalid action: {}", name,
00681                 ia.what());
00682         } catch (const std::exception& e) {
00683             std::println(stderr, "warning: execution '{}', dropped an action that threw: {}", name,
00684                 e.what());
00685         } catch (...) {
00686             std::println(stderr, "warning: execution '{}', dropped an action that threw an unknown type",
00687                 name);
00688         }
00689
00690         executing_action_ = false;
00691
00692         // Counted whether it ran cleanly, reported a dead binding, or threw: it came off the queue,
00693         // and the queue is what the notification is about.
00694         ++actions_run_;
00695
00696         // Whether the list emptied is read under the lock; the notification is raised without it.
00697         // on_finished is caller code and may call add_action(), which takes this same mutex - holding
00698         // it across the callback would deadlock the worker against itself.
00699         bool drained = false;
00700         {
00701             std::lock_guard<std::mutex> lock(action_mutex_);
00702             drained = action_list_.empty();
00703         }
00704
00705         if (drained) {
00706             notify_finished();
00707         }
00708     }
00709
00710     // Last, and part of the pass: an attachment is only ever reached through action_execute, which
00711     // is this function, so whoever drives this execution drives the ones attached to it.
00712     if (actuator_execute_.is_connected()) {
00713         actuator_execute_();
00714     }
00715 }
00716
00723 void notify_finished() {
00724     if (actions_run_ == 0 || !on_finished) {
00725         return;
00726     }
00727
00728     {
00729         // An action may queue another, so a pass that drained can leave more behind it. That is the
00730         // next batch, not the end of this one.
00731         std::lock_guard<std::mutex> lock(action_mutex_);
00732         if (!action_list_.empty()) {
00733             return;
00734         }
00735     }
00736
00737     on_finished();

```

```

00738 }
00739
00740 void execute() {
00741     // The results belong to this run: an execution that is run twice reports the second run's
00742     // results, not both runs' appended together.
00743     {
00744         std::lock_guard<std::mutex> lock(results_mutex_);
00745         results_.clear();
00746     }
00747
00748     execute_actions();
00749
00750     // After the drain, and after the callback it raised: from here on there is no more work, and a
00751     // caller polling is_running() should be told so. `running_` cannot serve instead - ~execution()
00752     // waits on it and may free this object the moment it reads false, so it has to stay the last
00753     // thing this worker touches.
00754     finishing_ = true;
00755
00756     std::println("execution '{}{}' finishing thread", name);
00757
00758     running_ = false;
00759 }
00760
00761 void loop() {
00762     for (;;) {
00763         {
00764             std::unique_lock<std::mutex> lock(action_mutex_);
00765
00766             // A bounded wait rather than a plain one: an attached execution has its own list and no way
00767             // to notify this condition variable, so the worker still has to look in on it periodically.
00768             action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00769                 [this] { return !action_list_.empty() || !started_; });
00770
00771             if (!started_ && action_list_.empty()) {
00772                 break;
00773             }
00774         }
00775         execute_actions();
00776     }
00777
00778     // A last pass over the attachments, not a last batch of this execution's own. The loop above
00779     // leaves only when this list is already empty, and add_action() refuses once stopped, so
00780     // nothing of ours can be waiting here and this pass reports nothing. What can be waiting is an
00781     // attached execution with actions queued: execute_actions() ends by driving them, and each
00782     // reports its own batch as it drains.
00783     execute_actions();
00784
00785     // Set here for the same reason execute() sets it, and in the same place: after the last pass,
00786     // because that pass still runs work - the attachments above - and a caller polling is_running()
00787     // may only be told the work is over once it is. `running_` cannot serve instead; it is the
00788     // handshake ~execution() waits on and has to stay the last thing this worker touches.
00789     finishing_ = true;
00790
00791     std::println("execution '{}{}' thread finished", name);
00792
00793     // Must stay last: ~execution() may free this object the moment it reads false.
00794     running_ = false;
00795 }
00796
00797 std::thread this_thread_;
00798
00799 std::atomic_bool started_ = {false};
00800 std::atomic_bool running_ = {false};
00801
00802 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00803 // may be queued before run()/start(), but not after stop().
00804 std::atomic_bool stopped_ = {false};
00805
00806 // action_list_, started_ and stopped_ are written by every thread that calls add_action() or
00807 // stop() and read by the worker; nothing touches them outside this mutex. action_cv_ is what
00808 // replaced the
00809 // spin in loop() and in stop().
00810 mutable std::mutex action_mutex_;
00811 std::condition_variable action_cv_;
00812
00813 std::list<std::function<typename actionT::result_type(void)>> action_list_;
00814
00823 using void_actuator = actuator<std::function<void(void)>>;
00824
00825 void_actuator actuator_execute_;
00826 void_actuator actuator_stop_;
00827
00841 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
00842
00856 void_actuator* attacher_execute_ = nullptr;
00857 void_actuator* attacher_stop_ = nullptr;
00858

```

```
00859 // std::vector cannot hold void type; use an arbitrary type e.g. int
00860 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00861     typename actionT::result_type>;
00862
00871 std::vector<typename resultT::type> results_;
00872
00879 mutable std::mutex results_mutex_;
00880
00888 std::atomic_bool collecting_results_ = {false};
00889
00903 std::atomic_bool finishing_ = {false};
00904
00913 std::atomic_bool executing_action_ = {false};
00914
00925 std::atomic_size_t actions_run_ = {0};
00926
00927 execution* other_this_;
00928 };
00929 } // namespace async
00930 } // namespace untangle
```


Index

- ~execution
 - untangle::async::execution< actionT >, [19](#)
- add
 - untangle::async::execution_poll, [26](#)
- add_action
 - untangle::async::execution< actionT >, [22](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [23](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [24](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- get
 - untangle::async::execution_poll, [27](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [27](#)
- is_busy
 - untangle::async::execution< actionT >, [21](#)
- is_running
 - untangle::async::execution< actionT >, [21](#)
 - untangle::async::execution_poll, [26](#)
- namespace untangle: functions, [13](#)
- remove
 - untangle::async::execution_poll, [26](#)
- results
 - untangle::async::execution< actionT >, [24](#)
- stop
 - untangle::async::execution< actionT >, [22](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [19](#)
 - add_action, [22](#)
 - attach, [23](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [24](#)
 - execution, [19](#)
 - is_busy, [21](#)
 - is_running, [21](#)
 - results, [24](#)
 - stop, [22](#)
- untangle::async::execution_poll, [25](#)
 - add, [26](#)
 - get, [27](#)
 - is_running, [26](#)
 - remove, [26](#)
- untangle::async::invalid_attachment, [27](#)
 - invalid_attachment, [27](#)
 - what, [28](#)
- what
 - untangle::async::invalid_attachment, [28](#)